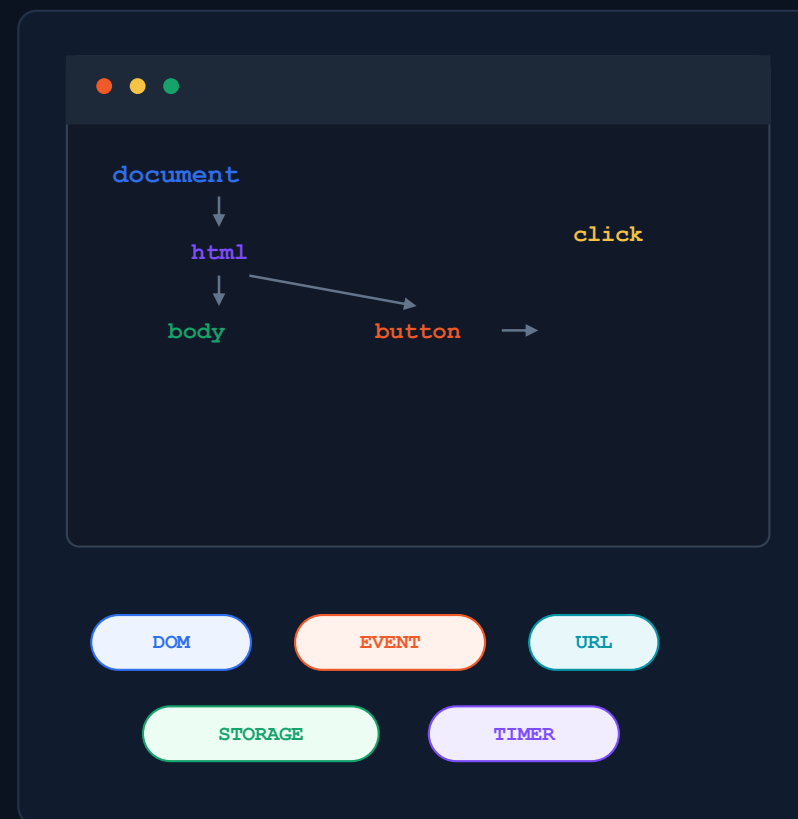


JavaScript в браузере — DOM, события и Browser APIs

управление документом, интерфейсом
и возможностями браузера

Дробышев Максим

старший разработчик · Яндекс Еда



Что успеем за 90 минут

01	Среда браузера и карта API	CONTEXT	0–12 мин
02	DOM: чтение и изменение документа	DOM	12–42 мин
03	События, делегирование и формы	EVENTS	42–70 мин
04	Навигация, хранение и таймеры	BROWSER	70–85 мин
05	Границы безопасности и выводы	SAFETY	85–90 мин

Без практических заданий: только объяснение, схемы и короткие демонстрации поведения API.

JavaScript + браузерное окружение

ECMAScript

Синтаксис и семантика языка

- значения и типы
- объекты и функции
- модули
- исключения

Один и тот же стандарт реализуют разные движки.

Движок

V8 · SpiderMonkey · JavaScriptCore

- парсит исходный код
- создаёт исполняемое представление
- оптимизирует горячие участки
- управляет памятью

Движок не обязан знать, что такое DOM.

Host environment

Браузер добавляет внешние возможности:

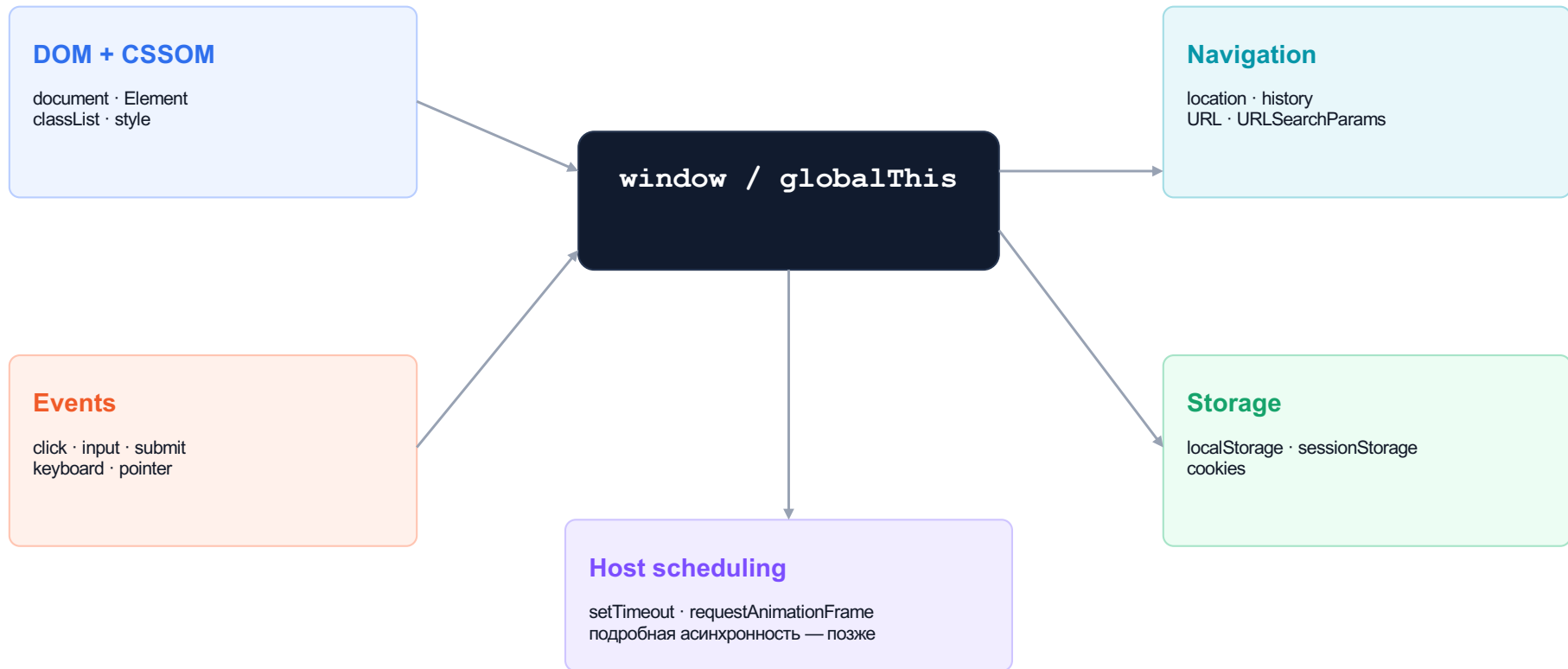
- document и DOM
- события пользователя
- навигацию и URL
- хранилища
- таймеры и разрешения

Именно host превращает язык в клиентскую платформу.

BOM — удобное учебное название набора объектов вокруг document; единой «спецификации BOM» нет.

Карта возможностей браузера

Не один API, а набор независимых подсистем



window, globalThis и глобальная область

console.js

```

1 window === globalThis      // true в окне
2 window.document === document // true
3 window.location.href
4 window.setTimeout
5
6 var legacy = 1;
7 window.legacy              // 1 в classic script
8
9 let local = 2;
10 window.local               // undefined

```

window

Глобальный объект контекста окна.

Содержит document, location, history, таймеры и другие API.

globalThis

Унифицированный доступ к глобальному объекту в разных окружениях.

Не привязывает код к браузеру.

Глобальные имена — общий ресурс

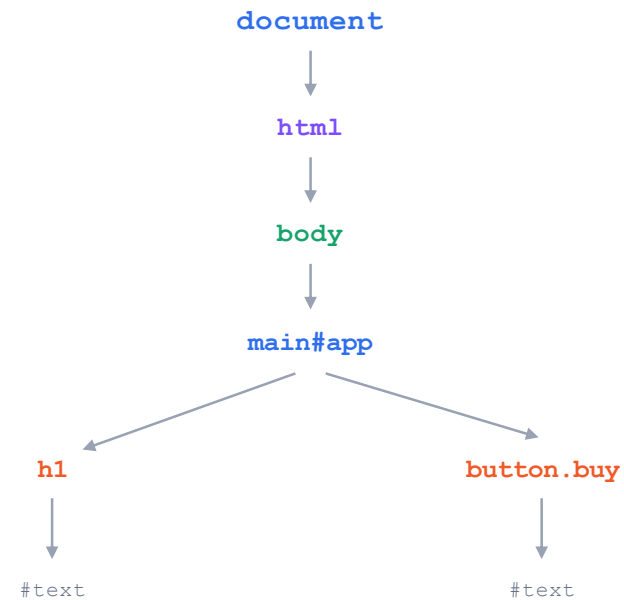
Слишком много глобальных переменных повышает риск конфликтов. ES-модули дают собственную область видимости и не добавляют bindings на window.

Практическая модель: браузерный API доступен через глобальный объект, но современный код живёт в модульной области.

HTML превращается в дерево объектов

index.html

```
1 <main id="app">
2   <h1>Каталог</h1>
3   <button class="buy">Купить</button>
4 </main>
```



DOM — живое представление

Браузер может исправить некорректный HTML, поэтому исходник и фактическое дерево иногда отличаются.

Node, Element и HTMLElement — разные уровни модели

Node

Любой узел дерева:
Document · Element · Text · Comment

childNodes видит текстовые и комментарные узлы.

Element

Только элементы разметки.

children, firstElementChild и closest работают с элементами, без Text-узлов.

EventTarget

базовая возможность подписки

Node

дерево: parent/child/sibling

Element

атрибуты, classList, selectors

HTMLElement

style, dataset, focus

HTMLElement

HTML-элементы получают общие свойства:
hidden, title, style, dataset, focus().

Специализированные типы

HTMLInputElement добавляет value/checked,
HTMLFormElement — elements и submit-методы.

Поиск элементов: CSS-селекторы как язык адресации

dom.js

```
1 const app = document.querySelector('#app');
2 const firstCard = app.querySelector('.card');
3 const cards = app.querySelectorAll('.card');
4
5 const byId = document.getElementById('app');
6
7 const active = document.querySelector(
8   '[data-state="active"]'
9 );
```

querySelector

Возвращает первый совпавший Element или null.

Можно вызывать у document и у конкретного элемента.

querySelectorAll

Возвращает статический NodeList всех совпадений.

Поддерживает forEach и итерацию.

getElementById

Самый прямой метод для уникального id. Работает только у document; не требует символа #.

Правило: начинайте с `querySelector/querySelectorAll`; специализированные методы используйте, когда их поведение действительно нужно.

NodeList и HTMLCollection: статический снимок и живой список

collections.js

```
1 const staticCards =  
2   document.querySelectorAll('.card');  
3  
4 const liveCards =  
5   document.getElementsByClassName('card');  
6  
7 document.body.append(newCard);  
8  
9 staticCards.length; // старое значение  
10 liveCards.length;  // обновилось
```

Static NodeList

querySelectorAll()

- фиксирует результат запроса
- изменения DOM не меняют коллекцию
- удобно безопасно перебирать
- можно преобразовать [...list]

Live HTMLCollection

getElementsByClassName()
getElementsByTagName()

- пересчитывается при изменении DOM
- может неожиданно менять length во время цикла

Практическое следствие

Если коллекция живёт во время удаления или добавления элементов, копируйте её в Array либо используйте статический querySelectorAll.

Навигация по дереву: родственники и поиск вверх

traversal.js

```
1 const button = event.target.closest('button');
2 if (!button) return;
3
4 const card = button.closest('.card');
5 const list = card.parentElement;
6
7 card.children;
8 card.firstChild;
9 card.nextElementSibling;
10
11 button.matches('[data-action="buy"]');
```

parentElement

ближайший родитель

children

дочерние элементы

closest(selector)

идёт вверх до совпадения

matches(selector)

проверяет текущий элемент

previous / next

соседи того же родителя

contains(node)

является ли узел потомком

Текст и разметка: `textContent`, `innerText`, `innerHTML`

`textContent`

Читает и записывает текст всех потомков.

- не интерпретирует HTML
- включает скрытый текст
- предсказуем для данных

```
message.textContent = userName;
```

`innerText`

Отражает визуально отображаемый текст.

- учитывает CSS и переносы
- может потребовать расчёта layout
- полезен для текста «как на экране»

`innerHTML`

Читает или заменяет HTML-разметку внутри элемента.

- запускает HTML-парсер
- создаёт новые узлы
- опасен для непроверенных данных

```
safe-text.js
```

```
title.textContent = '<strong>Привет</strong>'; // покажет строку, не тег
```

Разметка из строк: удобство, парсинг и риск XSS

unsafe.js

```
1 const name = input.value;
2
3 container.innerHTML =
4   '<p>Привет, ' + name + '</p>';
5
6 // Если name содержит разметку,
7 // браузер её распарсит.
8 // Непроверенный HTML — граница доверия.
```

Безопасный текст

Создавайте элементы и записывайте пользовательские значения через `textContent`.

Доверенный шаблон

Для статической известной разметки возможны `template` или `insertAdjacentHTML`. Данные всё равно должны быть экранированы/санитизированы.

Главная модель

Строка становится кодом разметки только в API, которое запускает HTML-парсер.

XSS — не «ошибка innerHTML», а результат смешивания данных и исполняемой разметки без границы доверия.

Атрибуты и свойства: исходная разметка и текущее состояние

input.html + js

```
1 <input value="Ada" checked>
2
3 const input = document.querySelector('input');
4
5 input.getAttribute('value'); // "Ada"
6 input.value;                // текущее значение
7
8 input.hasAttribute('checked'); // true
9 input.checked;               // текущее состояние
10
11 input.value = 'Grace';
```

Атрибут

Текстовая настройка в HTML.

getAttribute / setAttribute /
removeAttribute

Часто задаёт начальное значение.

Свойство

Поле JavaScript-объекта.

Может быть boolean, number,
объектом или функцией.

Описывает текущее состояние.

Синхронизация зависит от свойства

Некоторые пары отражаются друг в друга, другие расходятся после взаимодействия.
Для форм читайте value/checked как свойства.

Булевый атрибут checked="false" всё равно присутствует — значит, в разметке он истинный.

data-* хранит данные компонента, ARIA описывает доступность

component.html

```
1 <button
2   data-action="toggle-menu"
3   data-menu-id="main"
4   aria-expanded="false"
5   aria-controls="main-menu">
6   Меню
7 </button>
8
9 button.dataset.action; // "toggle-menu"
10 button.dataset.menuId; // "main"
```

dataset

data-menu-id ↔ dataset.menuId

Значения всегда строки. Хорошо подходит для идентификаторов и действий.

ARIA

aria-expanded сообщает assistive technology состояние раскрытия.

ARIA не создаёт поведение автоматически.

Состояние должно быть согласовано

Если меню открылось, код меняет и CSS-класс/data-state, и aria-expanded. Визуальное состояние и доступностное описание не должны расходиться.

Сначала используйте нативный HTML: button уже умеет фокус и клавиатуру. ARIA дополняет семантику, а не заменяет её.

classList: JavaScript меняет состояние, CSS решает внешний вид

state.js

```
1 const panel = document.querySelector('.panel');
2
3 panel.classList.add('panel--open');
4 panel.classList.remove('panel--loading');
5
6 panel.classList.toggle('panel--open');
7 panel.classList.toggle('panel--open', isOpen);
8
9 panel.classList.contains('panel--open');
10 panel.classList.replace('old', 'new');
```

Хорошее разделение ответственности

JavaScript выбирает состояние — open, loading, error. CSS описывает, как это состояние выглядит.

Почему лучше прямого style

Класс может менять много свойств, поддерживать :hover/@media и оставаться видимым в DevTools как единая причина.

toggle(name, force)

Второй аргумент задаёт желаемое состояние и избавляет от ручного if add/remove.

Inline style и CSSOM: записать значение и прочитать итог

styles.js

```
1 const box = document.querySelector('.box');
2
3 box.style.width = '240px';
4 box.style.backgroundColor = 'tomato';
5 box.style.setProperty('--progress', '72%');
6
7 box.style.width; // только inline-значение
8
9 const computed = getComputedStyle(box);
10 computed.width;
11 computed.getPropertyValue('--progress');
```

element.style

Записывает и читает атрибут style.

Имена свойств в camelCase; custom properties — через setProperty.

getComputedStyle

Возвращает вычисленные значения после каскада и наследования.

Результат доступен только для чтения.

Не путать specified и computed

В CSS записано width: 50%, а computed.width может стать 420px. Браузер уже учёл контейнер, каскад и единицы.

Чтение геометрии и запись стилей попеременно может заставить браузер преждевременно пересчитывать layout.

Создание элементов: структура отдельно, данные отдельно

render.js

```
1 function createCard(product) {  
2   const article = document.createElement('article');  
3   article.className = 'card';  
4   article.dataset.id = String(product.id);  
5  
6   const title = document.createElement('h2');  
7   title.textContent = product.title;  
8  
9   const button = document.createElement('button');  
10  button.type = 'button';  
11  button.textContent = 'Купить';  
12  
13  article.append(title, button);  
14  return article;  
15 }
```

createElement(tagName)

Создаёт Element, но ещё не вставляет его в документ.

textContent

Безопасно помещает данные пользователя как текст.

Свойства вместо строк

type, className, dataset и другие свойства дают явный API объекта.

Вставка, перемещение, замена и удаление узлов

mutations.js

```
1 list.append(card);           // в конец
2 list.prepend(card);         // в начало
3
4 card.before(label);          // перед узлом
5 card.after(separator);       // после узла
6
7 oldCard.replaceWith(newCard);
8 card.remove();
9
10 container.replaceChildren(...cards);
```

append / prepend

внутри контейнера

before / after

рядом с текущим узлом

replaceWith

замена самого узла

remove

удаление из дерева

replaceChildren

замена содержимого

Если вставить существующий узел в другое место, он переместится
— DOM не создаёт копию автоматически.

template, cloneNode и DocumentFragment

template.html + js

```
1 <template id="card-template">
2   <article class="card">
3     <h2></h2>
4     <button type="button">Купить</button>
5   </article>
6 </template>
7
8 const fragment = template.content.cloneNode(true);
9 fragment.querySelector('h2').textContent = title;
10 list.append(fragment);
```

<template>

Хранит неактивную DOM-структуру: содержимое не отображается и не исполняется до клонирования.

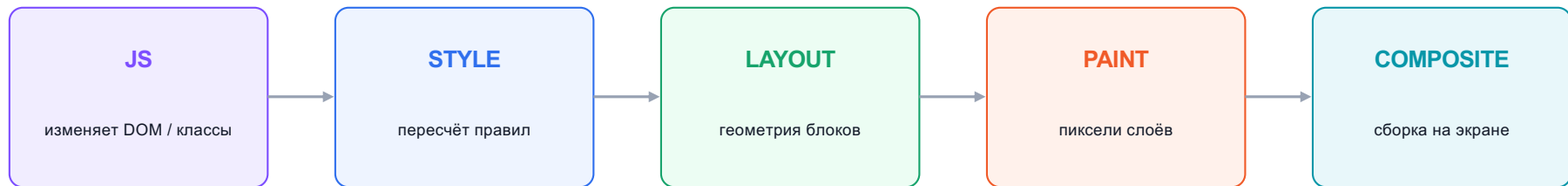
cloneNode(deep)

Создаёт копию узла. `deep=true` копирует потомков, но обработчики `addEventListener` не переносятся.

DocumentFragment

Лёгкий контейнер для группы узлов. При вставке в DOM переносится его содержимое, а не сам `fragment`.

Изменение DOM запускает работу рендеринга



Группируйте изменения

Сначала меняйте несколько классов/узлов, затем давайте браузеру отрисовать результат.

Не меряйте после каждой записи

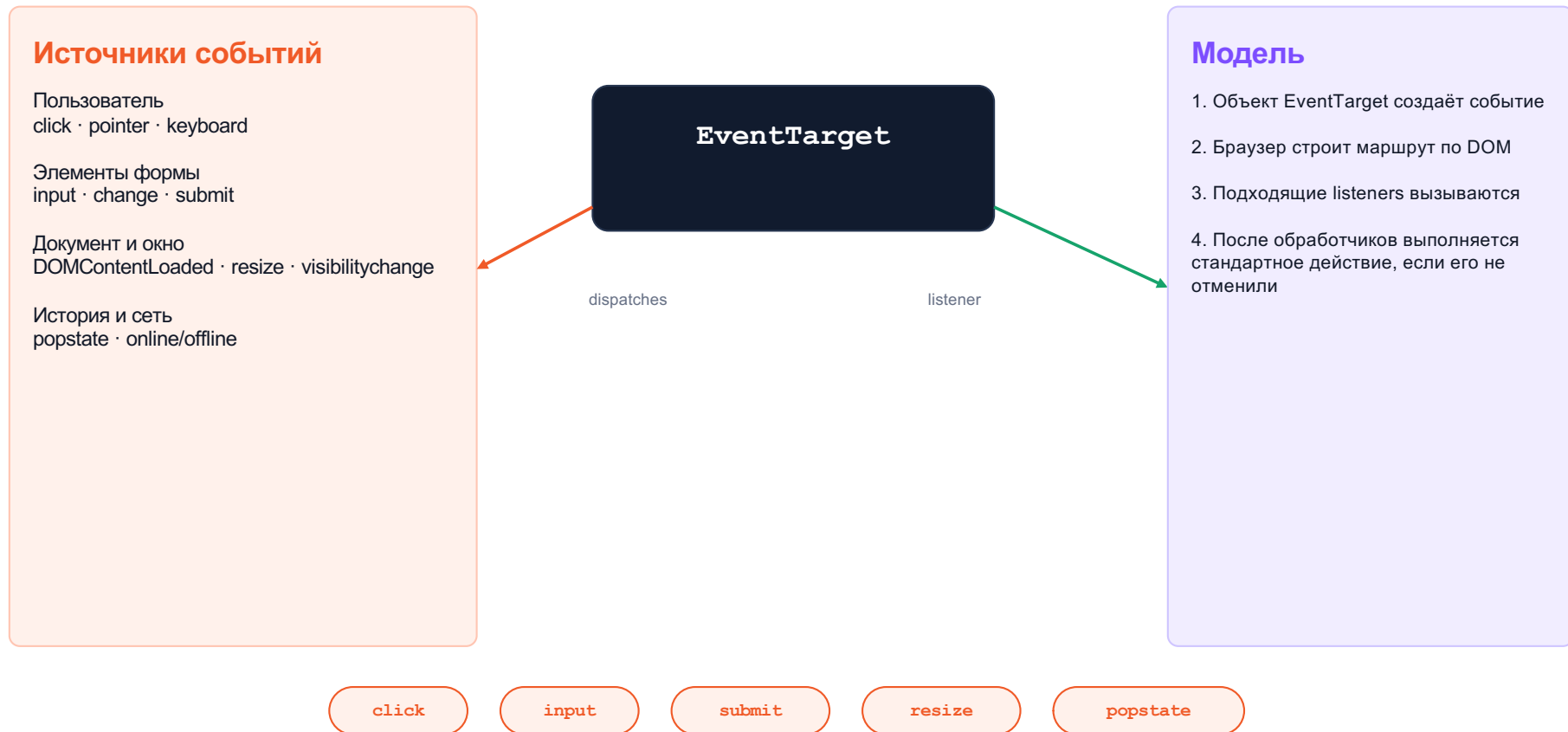
Чередование `style.width = ...` и `offsetWidth` заставляет синхронно завершать layout.

Используйте подходящий механизм

Для визуального состояния — классы; для движения — `transform/opacity`; для списка — пакетная вставка.

Оптимизация начинается не с микротрюков, а с понятной структуры обновления интерфейса.

Браузерная программа реагирует на события



addEventListener вместо inline onclick

```
events.js

1 const button = document.querySelector('.buy');
2
3 function handleClick(event) {
4   console.log(event.currentTarget);
5 }
6
7 button.addEventListener('click', handleClick);
8
9 // Позже:
10 button.removeEventListener('click', handleClick);
```

HTML onclick

Смешивает разметку и код.

Один атрибут — одна строка обработчика.

Сложнее управлять зависимостями.

property onclick

`button.onclick = fn`

Новое присваивание заменит предыдущее.

Иногда удобно для простого объекта.

addEventListener

Поддерживает несколько listeners, параметры `capture/once/passive/signal` и явное удаление. Это основной вариант для прикладного кода.

Чтобы `removeEventListener` сработал, нужна та же ссылка на функцию и тот же режим `capture`.

Жизненный цикл listener: once, passive, signal и удаление

listeners.js

```
1 const controller = new AbortController();
2
3 window.addEventListener('scroll', onScroll, {
4   passive: true,
5   signal: controller.signal,
6 });
7
8 button.addEventListener('click', handleOnce, {
9   once: true,
10 });
11
12 // удалить все listeners с этим signal
13 controller.abort();
```

once

автоматически удалить после первого вызова

passive

обещание не вызывать preventDefault; важно для scroll/touch

capture

слушать на фазе погружения

signal

снять подписку через AbortController

Анонимный callback нельзя точно удалить, если вы не сохранили ссылку или не используете signal.

Объект event содержит контекст произошедшего

handler.js

```
1 function handle(event) {  
2   event.type;      // "click"  
3   event.target;    // исходный узел  
4   event.currentTarget; // где висит listener  
5   event.timeStamp;  
6   event.isTrusted;  
7  
8   // PointerEvent  
9   event.clientX;  
10  event.button;  
11  
12  // KeyboardEvent  
13  event.key;  
14  event.code;  
15 }
```

Общее

type · target · currentTarget ·
timeStamp · cancelable ·
defaultPrevented

Специализация

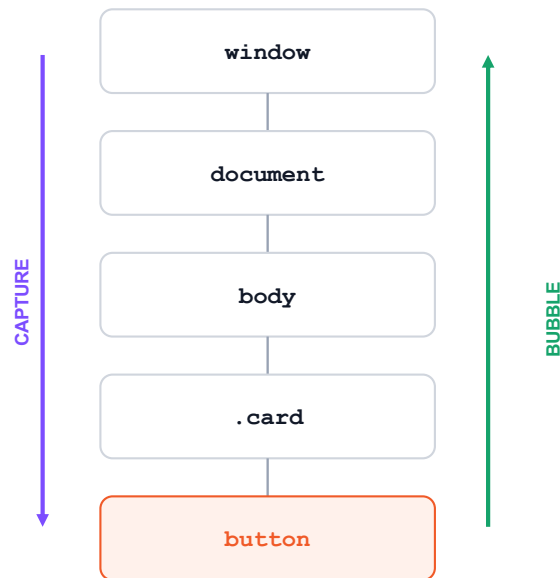
PointerEvent, KeyboardEvent,
InputEvent и SubmitEvent
добавляют свои данные.

this в обычной функции

В listener, переданном обычной function, this равен currentTarget. У arrow-функции собственного this нет — она берёт его из внешней области.

Для читаемого кода обычно используйте event.currentTarget явно,
а не полагайтесь на this.

Три фазы: capture → target → bubble



1 · Capture

Событие идёт от window к родителям target.
Listener должен быть зарегистрирован с `capture: true`.

2 · Target

Достигает исходного объекта.
target остаётся одним и тем же по всему маршруту.

3 · Bubble

Большинство UI-событий поднимаются от target к предкам.
На этом строится делегирование.

Не все события всплывают одинаково

`focus` и `blur` не bubble; для делегирования используют `focusin/focusout`. `mouseenter/leave` отличаются от `mouseover/out`.
Проверяйте контракт конкретного события.

Стандартное действие и распространение — разные механизмы

event-control.js

```
1 link.addEventListener('click', event => {
2   event.preventDefault();
3   // переход по href не произойдёт
4 });
5
6 child.addEventListener('click', event => {
7   event.stopPropagation();
8   // событие не пойдёт к следующим предкам
9 });
10
11 event.stopImmediatePropagation();
12 // также не вызовет следующие listeners
13 // на текущем EventTarget
```

preventDefault()

Отменяет действие браузера: переход по ссылке, отправку формы, drag и другие cancelable-действия. Не останавливает bubble.

stopPropagation()

Останавливает дальнейший маршрут события. Не отменяет стандартное действие и не выключает другие listeners на том же target.

Используйте осознанно

Частый stopPropagation ломает делегирование, аналитику и независимые компоненты. Предпочитайте фильтрацию по target.

Делегирование: один listener обслуживает динамический список

delegation.js

```
1 list.addEventListener('click', event => {
2   const button = event.target.closest(
3     '[data-action="remove"]'
4   );
5
6   if (!button || !list.contains(button)) return;
7
8   const item = button.closest('[data-item-id]');
9   removeItem(item.dataset.itemId);
10 });
```

Почему работает

click всплывает от вложенной иконки/кнопки к контейнеру списка.

Почему closest

target может быть span или svg внутри кнопки. closest находит ближайший элемент действия.

Преимущества

Меньше listeners, новые элементы работают автоматически, централизованная маршрутизация действий.

Делегирование привязывает поведение к стабильному контейнеру, а действия — к data-атрибутам.

Семейства событий: выбираем по смыслу взаимодействия

Pointer

pointerdown / pointerup
pointermove / pointerenter
click / contextmenu

Одна модель для мыши, пера и touch.

Keyboard

keydown / keyup
key · code · repeat

Используйте key, а не устаревший keyCode.

Focus

focus / blur
focusin / focusout

Фокус — часть доступности и навигации клавиатурой.

Input

beforeinput / input
change

input — каждое изменение, change — подтверждённое значение.

keyboard.js

```
field.addEventListener('keydown', event => {  
  if (event.key === 'Escape') closeDialog();  
});
```

pointer.js

```
canvas.addEventListener('pointermove', event => {  
  draw(event.clientX, event.clientY);  
});
```

Не делайте hover и pointer единственным способом получить важную информацию: интерфейс должен работать с клавиатурой и touch.

Формы — готовый браузерный API, а не набор случайных input

form.js

```
1 form.addEventListener('submit', event => {
2   event.preventDefault();
3
4   if (!form.checkValidity()) {
5     form.reportValidity();
6     return;
7   }
8
9   const data = new FormData(form);
10  data.get('email');
11  data.getAll('topic');
12 });
13
14 form.elements.email.value;
```

submit

Слушайте форму, а не только click кнопки: отправка возможна Enter и другими способами.

FormData

Собирает успешные поля по name. Поддерживает файлы и повторяющиеся имена.

Constraint Validation API

required, type, min/max и pattern дают validity, checkValidity() и reportValidity().

label, правильный type и нативная валидация дают больше доступности и поведения до написания JavaScript.

URL, location и History API

navigation.js

```
1 const url = new URL(location.href);
2 url.searchParams.set('page', '2');
3
4 history.pushState({ page: 2 }, '', url);
5
6 window.addEventListener('popstate', event => {
7   renderPage(event.state?.page ?? 1);
8 });
9
10 location.assign('/catalog');
11 location.replace('/login');
12 history.back();
```

URL

Парсинг адреса как объекта: origin, pathname, search, hash. URLSearchParams управляет query.

location

Текущий адрес и навигация. assign создаёт запись истории, replace заменяет текущую.

history

pushState/replaceState меняют URL без перезагрузки документа. Назад/вперёд сообщает popstate — приложение должно восстановить UI.

History API не загружает данные и не рендерит экран автоматически — оно синхронизирует состояние навигации.

Navigator, видимость страницы и разрешения

navigator

Данные и возможности агента:

- language / languages
- online
- clipboard
- geolocation
- mediaDevices
- share

Проверяйте наличие API, а не угадывайте браузер по userAgent.

Page Visibility

document.visibilityState
visibilitychange

Можно ставить на паузу обновления, когда вкладка скрыта, и возобновлять при возврате.

Не гарантирует, что пользователь смотрит на экран.

Разрешения и условия

Некоторые API требуют:

- HTTPS secure context
- явного разрешения пользователя
- пользовательского жеста
- доступного устройства

Отказ — нормальный результат, его нужно обрабатывать.

Хранение в браузере: Web Storage и cookies решают разные задачи

storage.js

```
1 const settings = { theme: 'dark' };
2
3 localStorage.setItem(
4   'settings',
5   JSON.stringify(settings)
6 );
7
8 const saved = JSON.parse(
9   localStorage.getItem('settings') ?? '{}'
10 );
11
12 sessionStorage.setItem('draft', text);
```

localStorage

Живёт между сессиями для origin.

Ключи и значения — строки.

Синхронный API: не храните большие объёмы и секреты.

sessionStorage

Изолирован в рамках вкладки/сессии.

Подходит для черновика или временного UI-состояния.

Cookies

Маленькие строки, которые браузер может отправлять серверу с HTTP-запросами. Для безопасности важны Secure, HttpOnly и SameSite.

Storage доступен JavaScript-коду origin: XSS превращает сохранённые токены и данные в добычу атакующего.

Таймеры и requestAnimationFrame — планирование через host

scheduling.js

```
1  const id = setTimeout(() => {  
2    showMessage();  
3  }, 500);  
4  
5  clearTimeout(id);  
6  
7  const intervalId = setInterval(updateClock, 1000);  
8  clearInterval(intervalId);  
9  
10 function frame(time) {  
11   renderAnimation(time);  
12   requestAnimationFrame(frame);  
13 }  
14 requestAnimationFrame(frame);
```

setTimeout

Запланировать callback не раньше задержки. Точное время не гарантируется.

setInterval

Повторяет callback, но длительная работа может приводить к дрейфу и накоплению.

requestAnimationFrame

Вызывает callback перед следующей отрисовкой и передаёт timestamp. Подходит для JS-анимации, синхронизированной с экраном.

Почему callback выполняется позже и как взаимодействуют очереди задач — тема следующей лекции об асинхронности.

Браузер мощный, но работает внутри песочницы

Same-origin policy

Страница не может произвольно читать DOM и данные другого origin. Протокол, хост и порт образуют границу.

Secure context

Чувствительные API обычно доступны только по HTTPS или localhost.

User permission

Камера, микрофон, геолокация и уведомления требуют контроля пользователя.

User activation

Clipboard, fullscreen и popup могут требовать непосредственного клика/жеста.

Нет произвольного доступа к ОС

Страница не читает файлы и процессы без явно предоставленного handle/выбора пользователя.

Данные ≠ код

textContent для текста, доверенная разметка отдельно, CSP и серверная валидация как дополнительные границы.

Проверка разрешений и обработка отказа — часть обычного UX, а не аварийная ветка.

Итоги занятия

Браузер превращает JavaScript из языка вычислений в платформу интерактивных приложений.